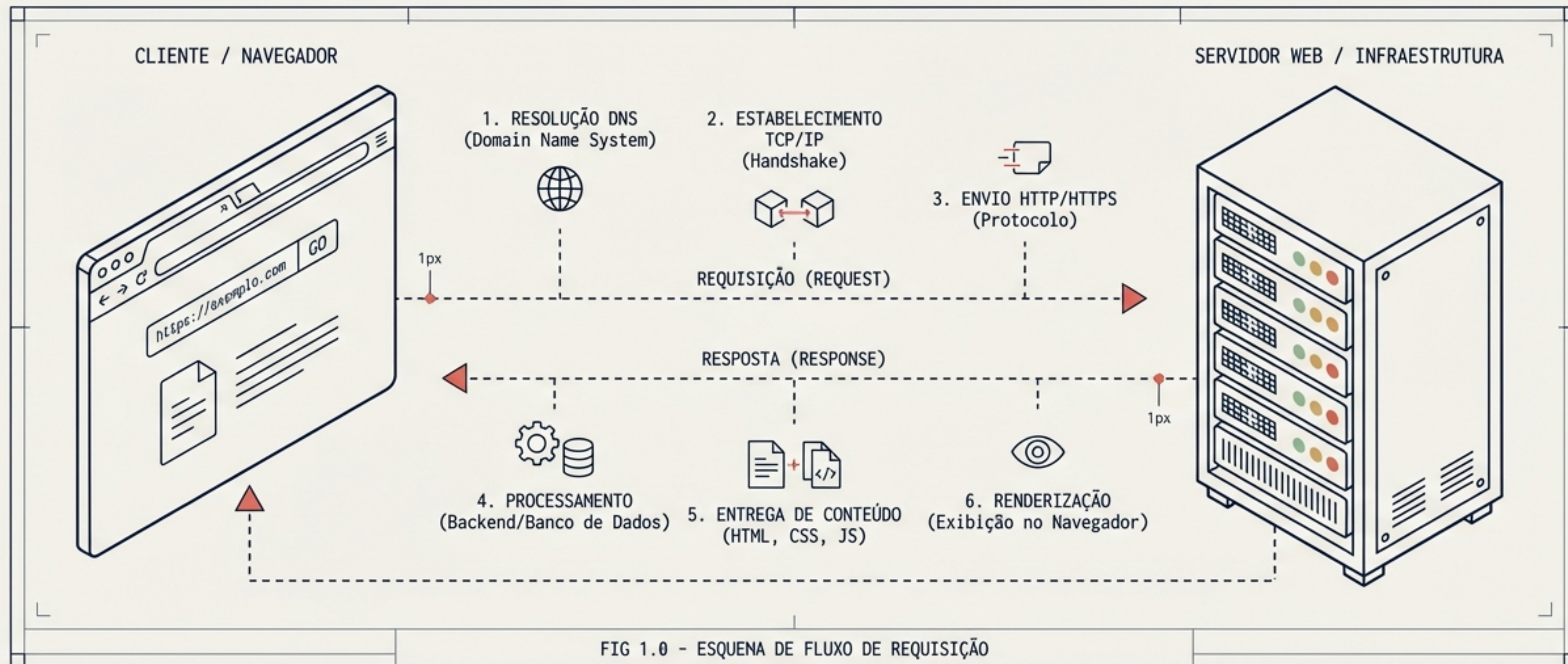


A anatomia de uma requisição web

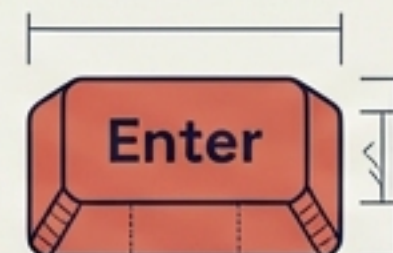
O que realmente acontece nos bastidores quando você aperta 'Enter'



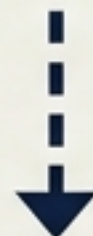
A mágica invisível da internet

A maioria das pessoas usa a web todos os dias sem pensar no que existe por baixo. Você digita um endereço, aperta **Enter**, e a página aparece. Mas a web não é mágica — é uma conversa estruturada.

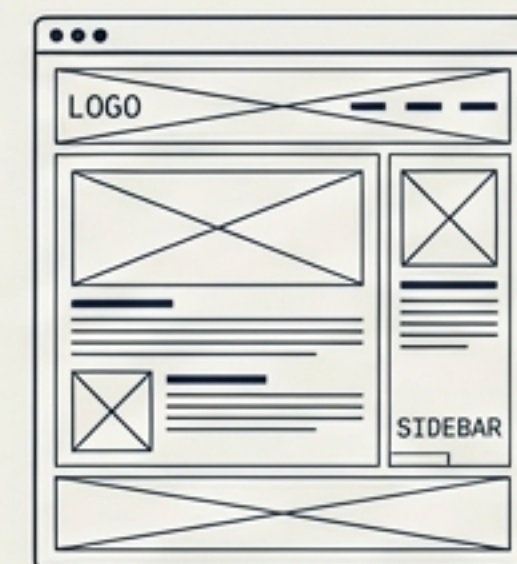
`https://www.exemplo.com/internet/como-funciona`



KEYCAP



CAIXA PRETA
DA INTERNET



HEADER

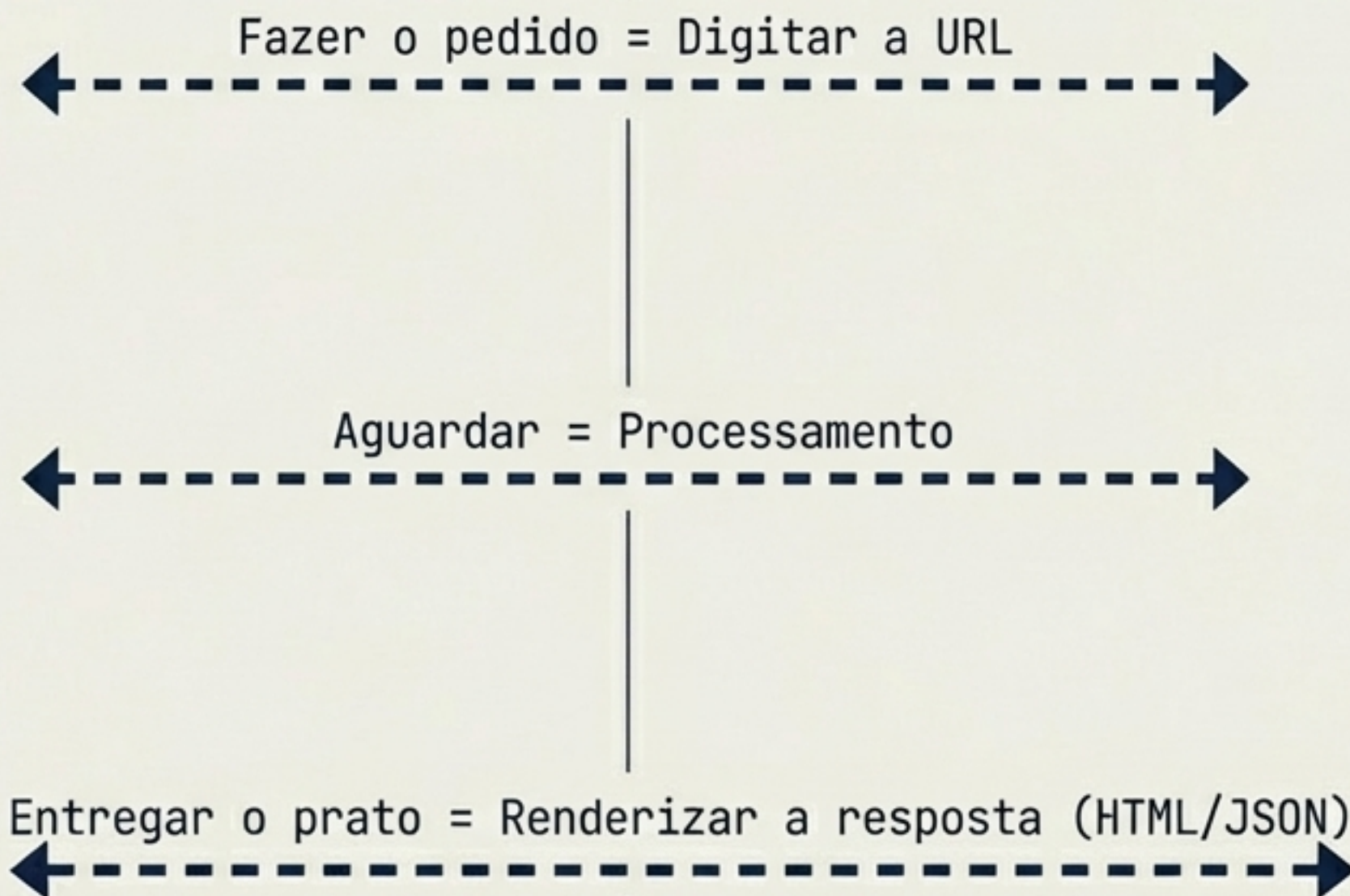
✓ SUCESSO

FOOTER

LAYOUT WIREFRAME

O Mundo Real

O Mundo Digital



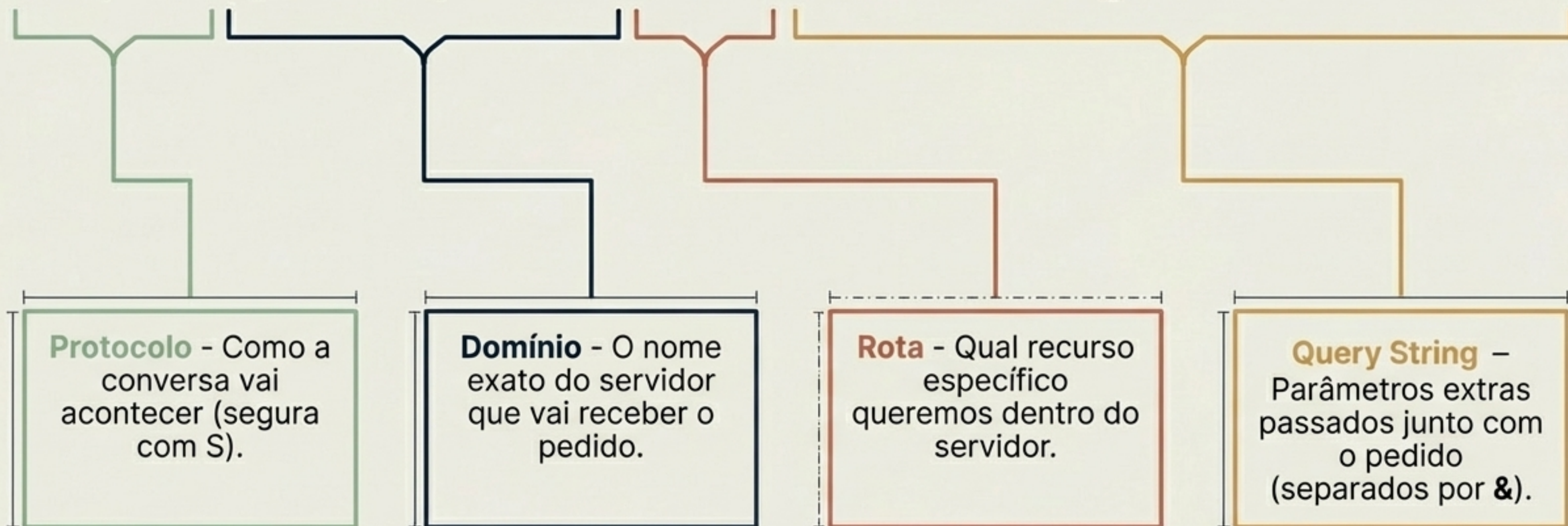
As regras da conversa: HTTP

Para o navegador e o servidor se entenderem, eles usam o HTTP (HyperText Transfer Protocol). É o conjunto universal de regras de comunicação da web. Toda vez que você acessa um site, um ou mais pedidos HTTP acontecem automaticamente.



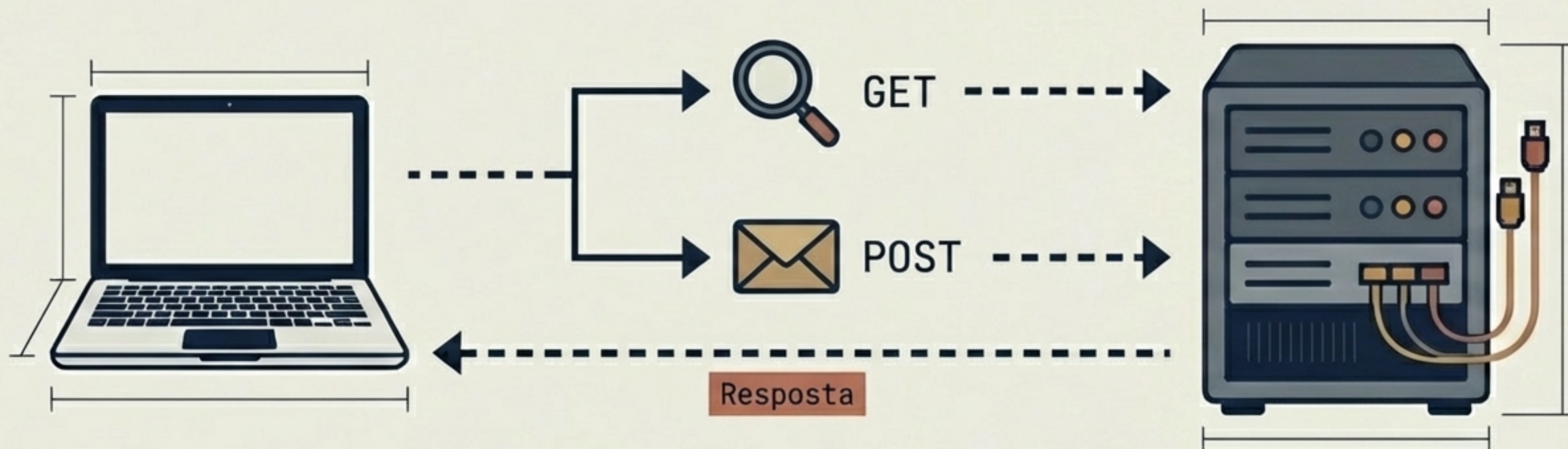
Desconstruindo o Pedido: A Estrutura da URL

`https://api.exemplo.com/clima?cidade=Aracaju&unidade=celsius`



A intenção do pedido

Quando o navegador faz um pedido, ele precisa declarar sua intenção através de um Método HTTP. Embora existam vários, a web moderna gira em torno de duas ações principais: buscar informações ou enviar informações.



A Batalha dos Métodos: GET vs. POST

	GET 	POST 
Ação Principal	Solicita um recurso do servidor.	Envia dados para o servidor processar.
Onde os dados viajam?	Na própria URL (Totalmente visíveis).	No corpo (body) da requisição (Ocultos da URL).
Quando usar?	Buscar dados, carregar páginas, pesquisar.	Criar contas, fazer login, upload de arquivos.
Exemplo Real	Uma pesquisa no Google (?q=javascript).	Digitar sua senha no Instagram.

O servidor sempre responde com um número

Além do conteúdo solicitado (uma imagem, uma página HTML), o servidor sempre devolve um código numérico de três dígitos. Esse código resume imediatamente o resultado da sua requisição, seguindo uma lógica universal.



O vocabulário das respostas HTTP



2xx - Sucesso

200 OK (Tudo certo, pedido entregue)

←-----

-----→

201 Created (Algo foi criado com sucesso)

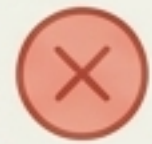
←-----

-----→



3xx - Redirecionamento

301 Redirect (O recurso mudou de lugar, siga para cá)



4xx - Erro do Cliente

400 Bad Request (Faltou algo no pedido)

←-----

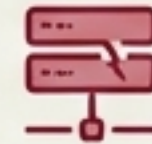
-----→

401/403 (Problemas de permissão/login)

←-----

-----→

404 Not Found (URL errada ou conteúdo removido)

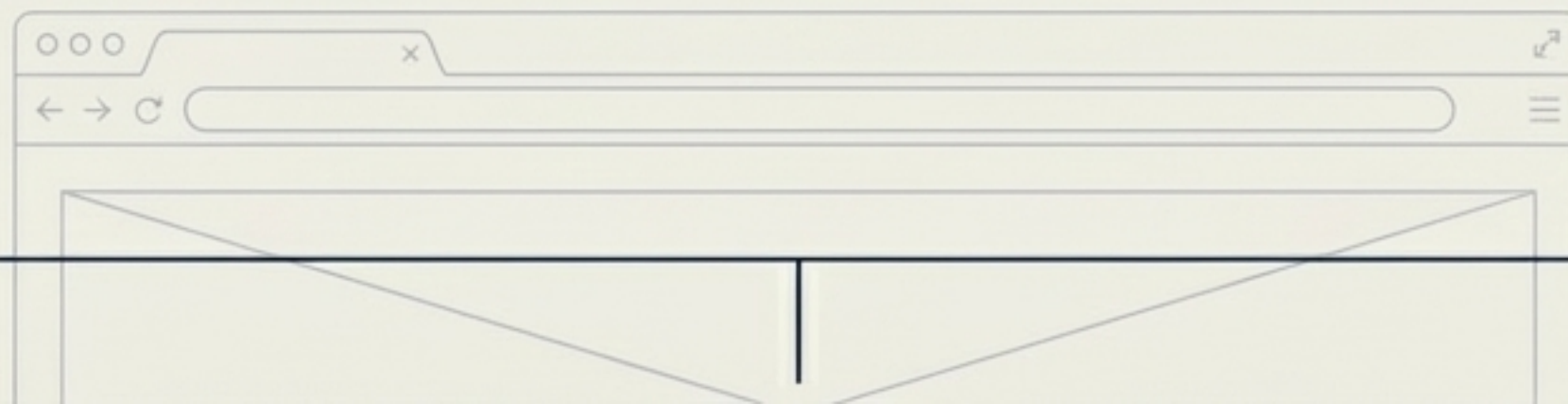
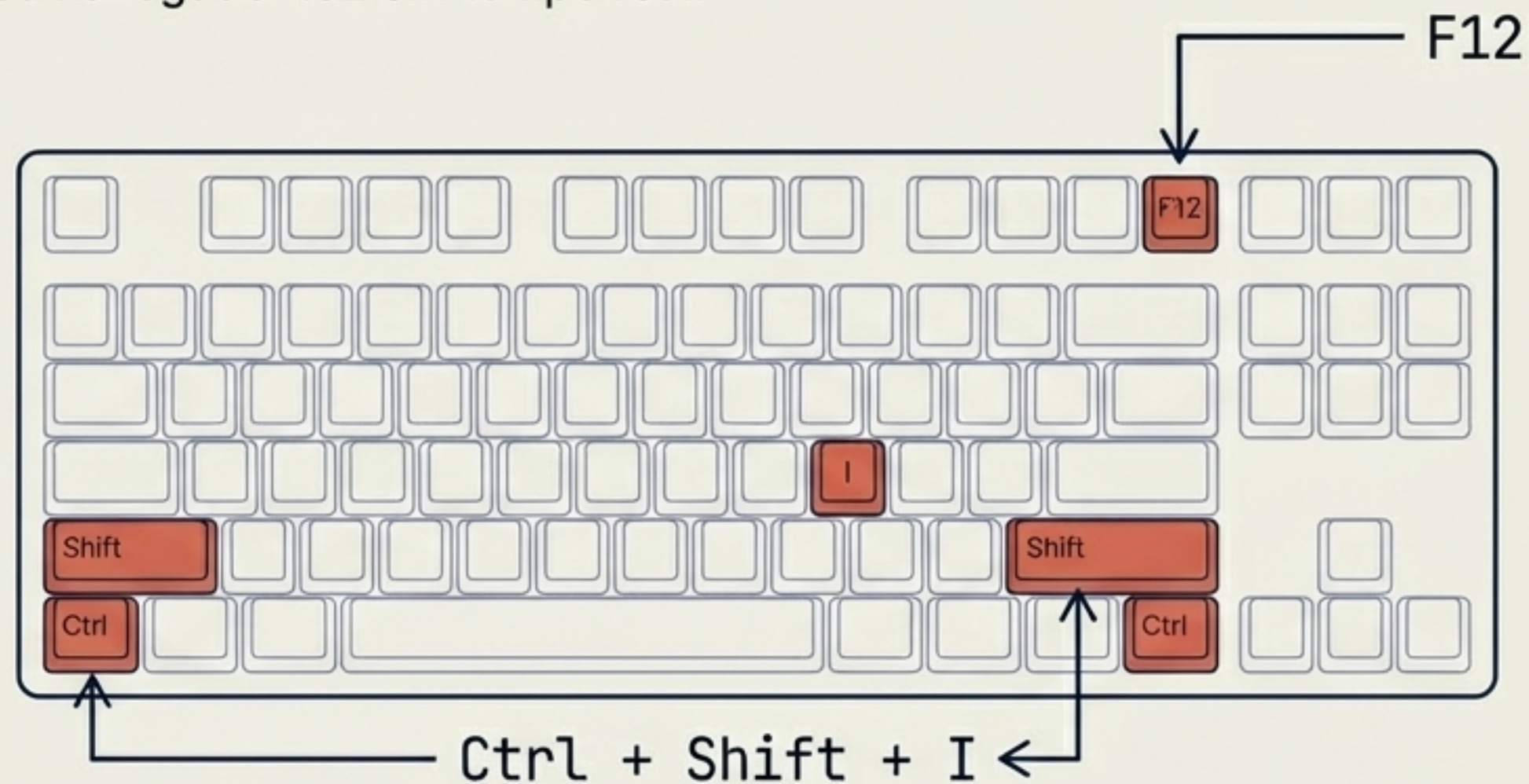


5xx - Erro do Servidor

500 Server Error (O problema é na cozinha deles, não no seu código)

Levantando o capô do navegador

Tudo o que discutimos até agora não é teoria invisível. A aba **Network (Rede)** do DevTools permite interceptar e inspecionar cada requisição HTTP que seu navegador faz em tempo real.



Navegando pela aba Network (Rede)

Network >> ⋮				
● 3				
1 Name	2 Method	Status	Type	4 Size
index.html	GET	200	document	1.2 KB
styles.css	GET	200	stylesheet	4.5 KB
api/data	GET	200	xhr	500 B

1. **Name:** O arquivo ou rota exata que foi solicitada ao servidor.

2. **Method:** A intenção da requisição (Aqui você verá os GETs e POSTs).

3. **Status:** O código de resposta (Esperamos ver muitos 200s aqui).

4. **Type & Size:** O formato do arquivo recebido e seu peso em dados.

Quando você clica em uma requisição na lista, você pode ler os **Headers** (cabeçalhos técnicos) e a **Response** (o conteúdo real que o servidor devolveu, como o clima abaixo).

```
fetch('https://wttr.in/Aracaju?format=3')  
  .then(r => r.text()).then(console.log)
```

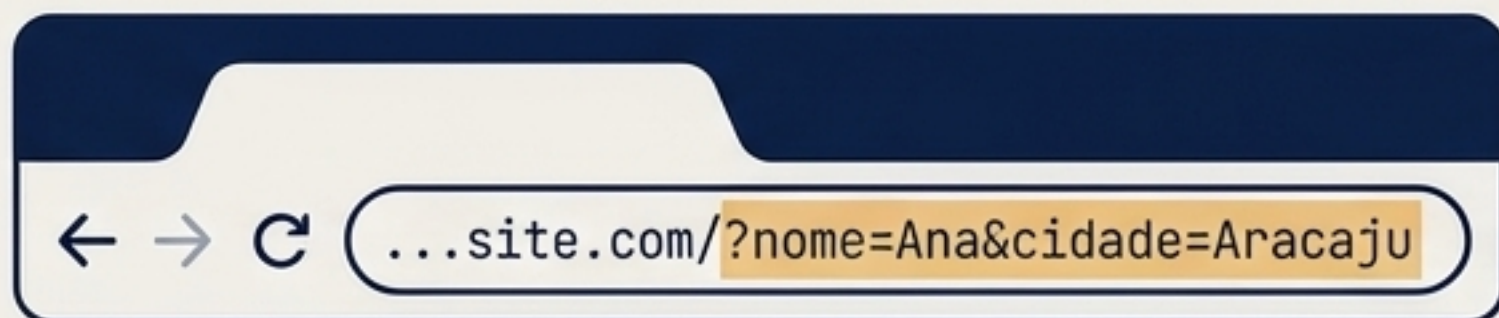


× Headers **Preview**

Aracaju: ☁ +29°C

Formulário com **GET**

```
<form method='get' action=''>
```



Os dados vazam visivelmente para a URL.

Formulário com **POST**

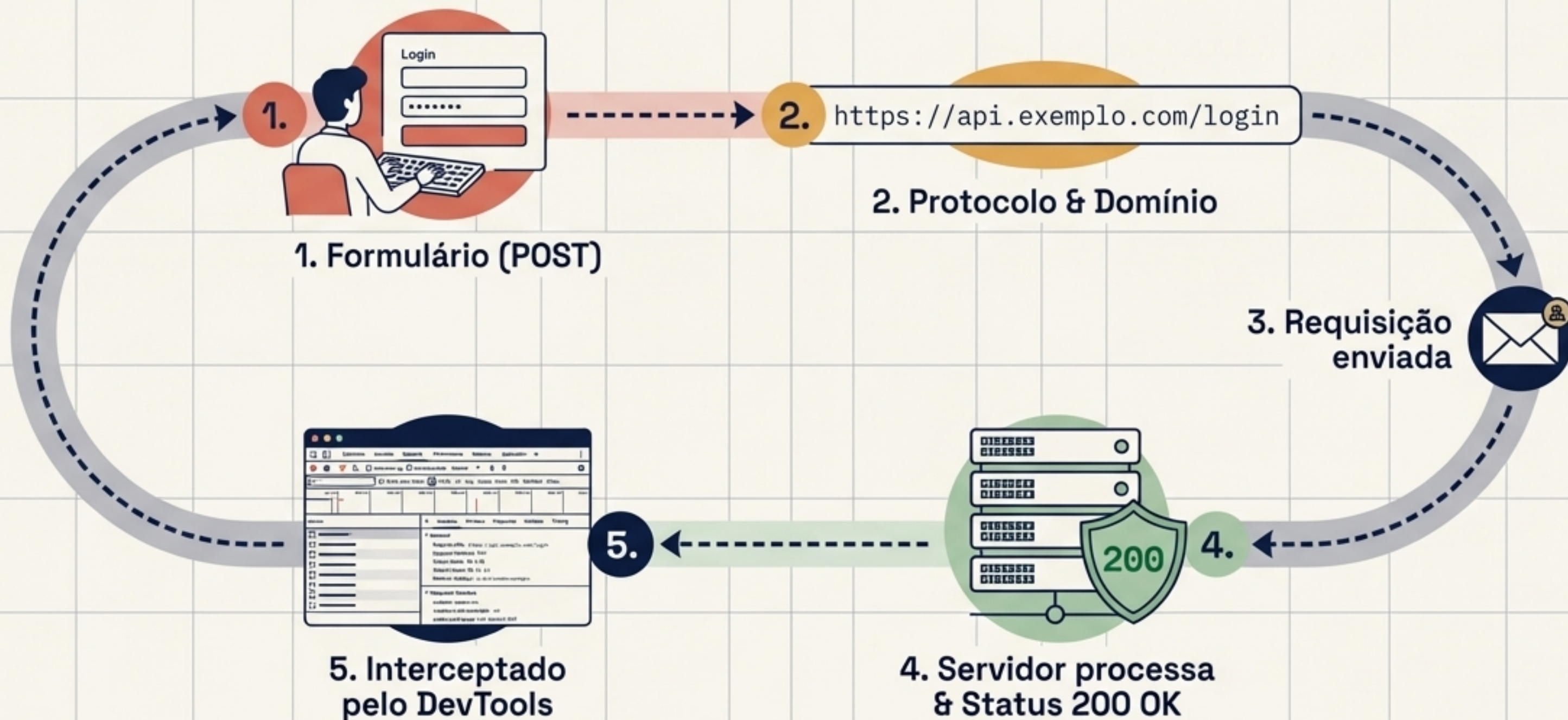
```
<form method='post' action=''>
```



A URL permanece limpa. Os dados viajam ocultos no Payload da requisição.

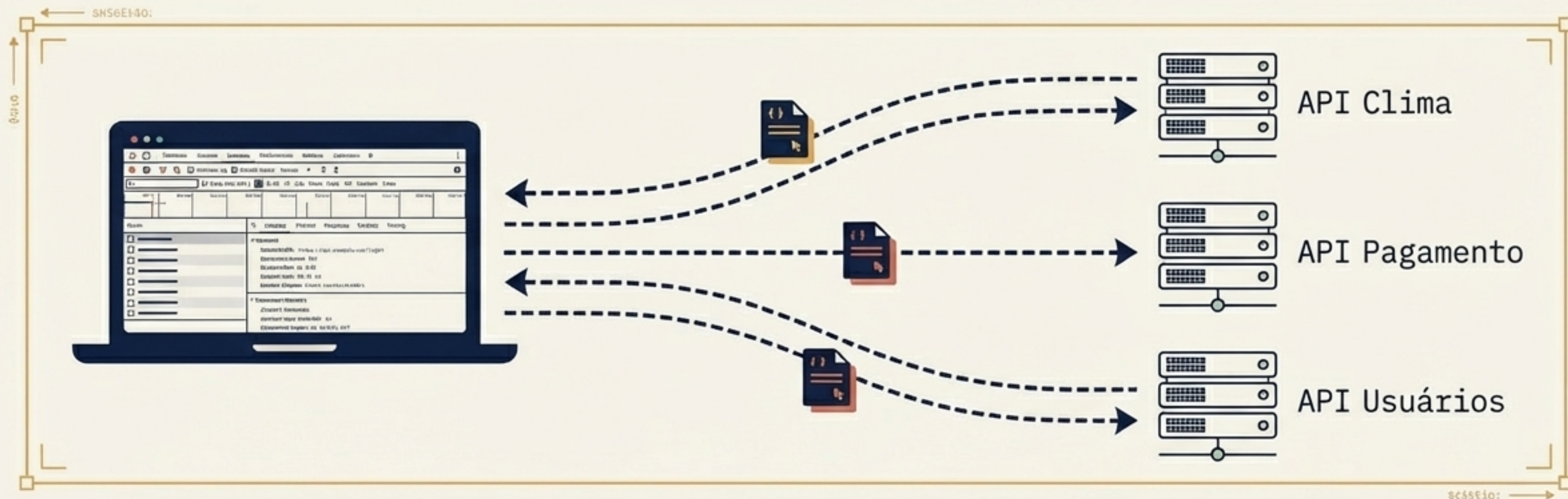
O ciclo de vida completo

Do seu teclado até o servidor e de volta à sua tela. Você agora domina as engrenagens essenciais da web.



O próximo passo...

Vimos como enviar nossos dados para o servidor. Mas e quando queremos fazer o caminho inverso automaticamente — buscar dados externos em tempo real sem recarregar a página?



 No próximo módulo: Dominando o `fetch()` e o consumo de APIs externas.